(an expression with multiple comparison operators is not legal PHP syntax), but logical operators are, and this does have some interesting effects:

```
$bool = getTrue() and getFalse();
var_dump($bool);
```

```
Output:

getTrue
getFalse
bool(true)
```

The result of what is essentially `$bool = true and false;` is `true`, yet the expression `true and false` is logically false. Why is that? We can see that both functions were called; there was no short-circuiting here. The reason we get `true` assigned is because the assignment operator has higher precedence than `and`. PHP first evaluates `$bool = getTrue()`, assigns to `$bool`, and only then goes on to evaluate `getTrue() and getFalse()`. The `&&` operator has a higher precedence than `=`, and gives the more intuitive answer:

```
$bool = getTrue() && getFalse();
var_dump($bool);
```

```
Output:
getTrue
getFalse
bool(false)
```

Be aware that the operators `or`, `xor`, and `and` run at a very low precedence (in fact the lowest precedence of all of PHP's operators) and can give weird results if this isn't taken into account. You are, of course, free to enforce higher precedence of these operators by using parentheses, which should also have the happy side effect of improving readability. The full breakdown of PHP's operator precedence and associativity can be found at https://www.php.net/manual/en/language.operators.precedence.php